

CoolDesk

PostgreSQL Database Schema

Production-Ready Design · NestJS Modular Monolith

A. Entity Breakdown

The schema is organised into 22 tables across six logical groups. Every table serves a single, well-defined purpose. No entity is overloaded.

Table	Purpose
virtual_customers	Customer identity anchor. Holds the VCID (UUID PK). No PII stored here — PII lives in jobs.
customer_phones	All phone numbers linked to a VCID. Many phones per VCID.
users	Owner, Office Staff, Technician. Single table with role enum.
dealers	Separate entity. Has own login, business details, region, and brand links.
dealer_brands	M:N join between dealers and brands.
brands	AC brands. Activatable/deactivatable. Colour-coded.
jobs	Central entity. Covers both installations and complaints. type + status drive lifecycle.
job_units	Child rows for multi-unit jobs. One row per AC unit.
job_assignments	Assignment history per job. One active row; superseded rows retained.
revisits	Child revisit records linked to a parent complaint job.
revisit_assignments	Per-revisit technician assignment history.
payments	1:1 with job. Records amount, method, status.
payment_methods	Owner-managed list of accepted payment methods.
job_timeline	Append-only audit log for every significant job event.
notifications	In-app notification store. One row per notification per recipient.
scheduling_conflicts	Log of acknowledged scheduling conflicts.
analytics_technician_daily	Precomputed daily rollup per technician for dashboard performance.

analytics_business_daily	Precomputed daily rollup for business-wide metrics.
analytics_brand_daily	Precomputed daily rollup per brand.
analytics_dealer_daily	Precomputed daily rollup per dealer.
system_config	Key-value store for owner-managed operational thresholds.
customer_reviews	Optional Module 19. Post-completion review links and ratings.

B. Full Database Schema (DDL)

All DDL is standard PostgreSQL 14+. Run sections in the order shown. Enum types must be created before the tables that reference them.

B.1 — Enums

```
-- =====
-- ENUMS
-- =====

CREATE TYPE user_role AS ENUM (
    'owner',
    'office_staff',
    'technician'
);

CREATE TYPE job_type AS ENUM (
    'installation',
    'complaint'
);

CREATE TYPE job_source AS ENUM (
    'direct',
    'via_dealer'
);

-- Installation statuses
CREATE TYPE installation_status AS ENUM (
    'pending_schedule', -- dealer-submitted only, before scheduling
    'scheduled',
    'assigned',
    'acknowledged',
    'in_transit',
    'in_process',
    'completed',
    'cancelled'
);

-- Complaint statuses
CREATE TYPE complaint_status AS ENUM (
```

```

    'new',
    'assigned',
    'acknowledged',
    'in_transit',
    'in_process',
    'resolved',
    'needs_revisit',
    'revisit_scheduled',
    'resolved_on_revisit',
    'cancellation_requested',
    'cancelled'
);

-- Combined status stored as TEXT (enforced per type in service layer)

CREATE TYPE payment_status AS ENUM (
    'collected',
    'pending'
);

CREATE TYPE visit_outcome AS ENUM (
    'on_time',
    'late',
    'no_show',
    'rescheduled'
);

CREATE TYPE revisit_reason AS ENUM (
    'part_unavailable',
    'customer_not_home',
    'issue_recurring',
    'further_diagnosis_required',
    'custom'
);

CREATE TYPE cancellation_request_status AS ENUM (
    'pending',
    'approved',
    'rejected'
);

CREATE TYPE timeline_event_type AS ENUM (
    'status_transition',
    'status_undo',
    'status_rollback',
    'assignment',
    'reassignment',
    'scheduling_conflict_ack',
    'payment_recorded',
    'payment_edited',
    'payment_void',

```

```

'payment_retained',
'cancellation',
'cancellation_request',
'cancellation_approved',
'cancellation_rejected',
'reopened',
'customer_linked',
'customer_link_dismissed',
'note_added',
'revisit_created',
'revisit_scheduled',
'rollback_payment_voided'
);

CREATE TYPE notification_event AS ENUM (
    'dealer_job_submitted',
    'job_assigned',
    'cancellation_request_submitted',
    'cancellation_request_outcome',
    'job_unacknowledged',
    'no_show_flagged',
    'repeat_complaint_detected',
    'frequent_complaint_detected',
    'third_revisit_reached'
);

CREATE TYPE review_mode AS ENUM (
    'off',
    'optional',
    'mandatory'
);

```

B.2 — Customer Identity

```

-- _____
-- VIRTUAL CUSTOMER IDENTITY
-- _____

CREATE TABLE virtual_customers (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    -- No PII here — name/address embedded in jobs.
    -- Flags computed by the repeat-complaint engine.
    is_frequent        BOOLEAN            NOT NULL DEFAULT FALSE,
    frequent_flagged_at TIMESTAMPTZ,
    created_at          TIMESTAMPTZ       NOT NULL DEFAULT NOW(),
    updated_at          TIMESTAMPTZ       NOT NULL DEFAULT NOW()
);

CREATE TABLE customer_phones (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

vcid          UUID    NOT NULL REFERENCES virtual_customers(id),
phone         TEXT     NOT NULL,
is_primary    BOOLEAN NOT NULL DEFAULT FALSE,
linked_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
linked_by     UUID,      -- FK to users.id (NULL if auto-linked)
UNIQUE (vcid, phone)
);

-- Fast phone-number lookup for VCID resolution
CREATE INDEX idx_customer_phones_phone ON customer_phones (phone);
CREATE INDEX idx_customer_phones_vcid  ON customer_phones (vcid);

```

B.3 — Users & Dealers

```

-- _____
-- USERS
-- _____

CREATE TABLE users (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  role        user_role     NOT NULL,
  full_name   TEXT          NOT NULL,
  email       TEXT          UNIQUE NOT NULL,
  password_hash TEXT       NOT NULL,
  is_active   BOOLEAN       NOT NULL DEFAULT TRUE,
  created_by  UUID          REFERENCES users(id),  -- NULL for owner seed
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);

-- _____
-- DEALERS
-- _____

CREATE TABLE dealers (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  business_name TEXT       NOT NULL,
  contact_name TEXT       NOT NULL,
  email       TEXT          UNIQUE NOT NULL,
  password_hash TEXT       NOT NULL,
  region      TEXT,
  is_active   BOOLEAN       NOT NULL DEFAULT TRUE,
  created_by  UUID          NOT NULL REFERENCES users(id),
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);

-- _____
-- BRANDS
-- _____

```

```

CREATE TABLE brands (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT          NOT NULL UNIQUE,
  colour_hex  CHAR(6),      -- e.g. '2E75B6'
  is_active   BOOLEAN NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

-- Brands associated with a dealer (M:N)
CREATE TABLE dealer_brands (
  dealer_id   UUID NOT NULL REFERENCES dealers(id),
  brand_id    UUID NOT NULL REFERENCES brands(id),
  PRIMARY KEY (dealer_id, brand_id)
);

```

B.4 — Jobs (Core Entity)

```

-- =====
-- JOBS
-- =====

CREATE TABLE jobs (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),

  -- Mandatory classification fields (NOT NULL enforced)
  type        job_type      NOT NULL,
  source       job_source    NOT NULL,
  status       TEXT          NOT NULL, -- validated against correct enum per
  type in service layer

  -- Brand (mandatory)
  brand_id     UUID          NOT NULL REFERENCES brands(id),

  -- Dealer link (populated only when source = 'via_dealer')
  dealer_id    UUID          REFERENCES dealers(id),

  -- Customer identity (VCID always present)
  vcid         UUID          NOT NULL REFERENCES virtual_customers(id),

  -- Embedded customer PII
  customer_name TEXT          NOT NULL,
  address       TEXT          NOT NULL,

  -- Flags derived from VCID history
  is_repeat     BOOLEAN       NOT NULL DEFAULT FALSE,
  is_frequent   BOOLEAN       NOT NULL DEFAULT FALSE,
  is_chronic    BOOLEAN       NOT NULL DEFAULT FALSE, -- 3+ revisits on this job

```

```

-- Scheduling
scheduled_at    TIMESTAMPTZ,
actual_arrival  TIMESTAMPTZ,
visit_outcome   visit_outcome,
late_by_minutes INT          CHECK (late_by_minutes >= 0),

-- Complaint-specific (NULL on installations)
issue_description TEXT,

-- Dealer cancellation request
cancellation_request_status cancellation_request_status,
cancellation_request_reason TEXT,
cancellation_request_at    TIMESTAMPTZ,
cancellation_decided_by    UUID          REFERENCES users(id),
cancellation_decided_at    TIMESTAMPTZ,

-- Cancellation/closure metadata
cancelled_at    TIMESTAMPTZ,
cancelled_by    UUID          REFERENCES users(id),
cancellation_reason TEXT,

-- Optimistic locking
version        INT          NOT NULL DEFAULT 0,

created_by     UUID          REFERENCES users(id),
created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),

-- — Data integrity rules —————
-- dealer_id is required when source is via_dealer
CONSTRAINT chk_dealer_source
    CHECK (source <> 'via_dealer' OR dealer_id IS NOT NULL),

-- Cancellation metadata must co-exist
CONSTRAINT chk_cancel_meta
    CHECK (cancelled_at IS NULL OR (cancelled_by IS NOT NULL AND
cancellation_reason IS NOT NULL)),

-- Soft-delete flag (records never physically removed)
is_deleted     BOOLEAN      NOT NULL DEFAULT FALSE
);

-- — Job unit entries (multi-unit support) —————
CREATE TABLE job_units (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id      UUID          NOT NULL REFERENCES jobs(id),
    model       TEXT          NOT NULL,
    unit_type   TEXT          NOT NULL,
    -- Installation-specific
    num_units   SMALLINT CHECK (num_units > 0),

```

```

-- Complaint-specific
fault_description TEXT,
created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_job_units_job ON job_units (job_id);

```

B.5 — Assignments

```

-- =====
-- JOB ASSIGNMENTS
-- =====
-- One row = one assignment event. is_active = TRUE marks current.
-- History preserved by never deleting superseded rows.

CREATE TABLE job_assignments (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id      UUID          NOT NULL REFERENCES jobs(id),
  technician_id UUID       NOT NULL REFERENCES users(id),
  is_active   BOOLEAN       NOT NULL DEFAULT TRUE,
  assigned_by UUID          NOT NULL REFERENCES users(id),
  assigned_at TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  acknowledged_at TIMESTAMPTZ -- populated when technician acknowledges
  unassigned_at TIMESTAMPTZ   -- populated when replaced

  -- Only one active assignment allowed per job (enforced by partial index):
);

CREATE UNIQUE INDEX idx_job_assignments_active
  ON job_assignments (job_id)
  WHERE is_active = TRUE;

CREATE INDEX idx_job_assignments_technician ON job_assignments (technician_id);
CREATE INDEX idx_job_assignments_job       ON job_assignments (job_id);

```

B.6 — Revisits

```

-- =====
-- REVISITS
-- =====

CREATE TABLE revisits (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id      UUID          NOT NULL REFERENCES jobs(id),
  sequence_number SMALLINT   NOT NULL,      -- 1, 2, 3 ... per parent job
  reason      revisit_reason NOT NULL,
  custom_reason TEXT,        -- populated when reason = 'custom'
  scheduled_at TIMESTAMPTZ,
  actual_arrival TIMESTAMPTZ,

```

```

visit_outcome    visit_outcome,
late_by_minutes INT          CHECK (late_by_minutes >= 0),
outcome          TEXT        CHECK (outcome IN ('resolved', 'needs_revisit')),
notes            TEXT,
created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),

CONSTRAINT chk_custom_reason
CHECK (reason <> 'custom' OR custom_reason IS NOT NULL),

UNIQUE (job_id, sequence_number)
);

CREATE INDEX idx_revisits_job ON revisits (job_id);

-- Revisit assignments (same pattern as job_assignments)
CREATE TABLE revisit_assignments (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  revisit_id   UUID          NOT NULL REFERENCES revisits(id),
  technician_id UUID        NOT NULL REFERENCES users(id),
  is_active    BOOLEAN       NOT NULL DEFAULT TRUE,
  assigned_by  UUID          NOT NULL REFERENCES users(id),
  assigned_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  acknowledged_at TIMESTAMPTZ,
  unassigned_at TIMESTAMPTZ
);

CREATE UNIQUE INDEX idx_revisit_assignments_active
ON revisit_assignments (revisit_id)
WHERE is_active = TRUE;

```

B.7 — Payments

```

-- _____
-- PAYMENTS
-- _____

CREATE TABLE payment_methods (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT          NOT NULL UNIQUE,    -- e.g. 'Cash', 'Bank Transfer'
  is_active    BOOLEAN       NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE payments (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id      UUID          NOT NULL UNIQUE REFERENCES jobs(id),  -- 1:1
  amount      NUMERIC(12, 2) NOT NULL CHECK (amount >= 0),
  payment_method_id UUID    NOT NULL REFERENCES payment_methods(id),
  status      payment_status NOT NULL DEFAULT 'pending',

```

```

    recorded_by      UUID          NOT NULL REFERENCES users(id),      --
    technician
    recorded_at      TIMESTAMPTZ    NOT NULL DEFAULT NOW(),
    -- Populated when owner reverses status and chooses void
    voided_at        TIMESTAMPTZ,
    voided_by        UUID          REFERENCES users(id),
    void_reason      TEXT,
    -- Populated when owner reverses status and chooses retain
    retained_note     TEXT,        -- 'Recorded on prior completion'
    -- Optimistic locking
    version          INT            NOT NULL DEFAULT 0,
    updated_at       TIMESTAMPTZ    NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_payments_job ON payments (job_id);

```

B.8 — Timeline (Audit Log)

```

-- =====
-- JOB TIMELINE (append-only audit trail)
-- =====

CREATE TABLE job_timeline (
    id            UUID            PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id        UUID            NOT NULL REFERENCES jobs(id),
    event_type    timeline_event_type NOT NULL,

    -- Actor (one of the following will be populated)
    actor_user_id UUID            REFERENCES users(id),
    actor_dealer_id UUID          REFERENCES dealers(id),

    -- Generic before/after payload (flexible, avoids wide nullable columns)
    previous_value JSONB,        -- e.g. {"status": "in_process"}
    new_value      JSONB,        -- e.g. {"status": "completed"}

    -- Human-readable reason (mandatory for rollbacks, cancellations, voids)
    reason         TEXT,

    -- Related entity references
    related_job_id UUID            REFERENCES jobs(id),        -- for scheduling conflict
    cross-ref
    revisit_id     UUID            REFERENCES revisits(id),

    -- Notes (for note_added events)
    note_text      TEXT,

    occurred_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
    -- No updates ever. Rows are immutable once inserted.
);

```

```
CREATE INDEX idx_timeline_job          ON job_timeline (job_id, occurred_at DESC);
CREATE INDEX idx_timeline_event_type  ON job_timeline (event_type);
CREATE INDEX idx_timeline_actor_user  ON job_timeline (actor_user_id);
```

B.9 — Notifications

```
-- =====
-- NOTIFICATIONS
-- =====

CREATE TABLE notifications (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  event_type  notification_event NOT NULL,
  job_id      UUID          REFERENCES jobs(id),

  -- Recipient (one of the following)
  recipient_user_id  UUID  REFERENCES users(id),
  recipient_dealer_id UUID  REFERENCES dealers(id),

  is_read          BOOLEAN NOT NULL DEFAULT FALSE,
  read_at          TIMESTAMPTZ,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CONSTRAINT chk_recipient
    CHECK (recipient_user_id IS NOT NULL OR recipient_dealer_id IS NOT NULL)
);

CREATE INDEX idx_notifications_recipient_user
  ON notifications (recipient_user_id, is_read, created_at DESC);
CREATE INDEX idx_notifications_recipient_dealer
  ON notifications (recipient_dealer_id, is_read, created_at DESC);
CREATE INDEX idx_notifications_job ON notifications (job_id);
```

B.10 — System Config

```
-- =====
-- SYSTEM CONFIGURATION
-- =====

CREATE TABLE system_config (
  key      TEXT  PRIMARY KEY,
  value    TEXT  NOT NULL,
  updated_by  UUID  NOT NULL REFERENCES users(id),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Seed defaults
INSERT INTO system_config (key, value, updated_by)
VALUES
```

```

('repeat_complaint_window_days', '30', '<owner-uuid>'),
('frequent_complaint_threshold', '3', '<owner-uuid>'),
('frequent_complaint_window_days', '90', '<owner-uuid>'),
('punctuality_grace_period_mins', '15', '<owner-uuid>'),
('customer_review_mode', 'off', '<owner-uuid>'),
('undo_window_seconds', '60', '<owner-uuid>');
-- (owner uuid substituted at deploy time)

```

B.11 — Analytics (Precomputed)

```

-- =====
-- PRECOMPUTED ANALYTICS TABLES
-- =====
-- Populated by background triggers / workers on each job event.
-- Dashboard queries hit these tables, not jobs directly.

CREATE TABLE analytics_technician_daily (
  id                UUID      PRIMARY KEY DEFAULT gen_random_uuid(),
  technician_id     UUID      NOT NULL REFERENCES users(id),
  date              DATE      NOT NULL,
  jobs_assigned     INT       NOT NULL DEFAULT 0,
  installations_completed INT   NOT NULL DEFAULT 0,
  complaints_resolved INT   NOT NULL DEFAULT 0,
  revisit_count     INT       NOT NULL DEFAULT 0,
  no_show_count     INT       NOT NULL DEFAULT 0,
  on_time_count     INT       NOT NULL DEFAULT 0,
  late_count        INT       NOT NULL DEFAULT 0,
  total_payment_collected NUMERIC(14,2) NOT NULL DEFAULT 0,
  pending_payment_total NUMERIC(14,2) NOT NULL DEFAULT 0,
  avg_resolution_mins NUMERIC(10,2),
  avg_acknowledgment_mins NUMERIC(10,2),
  first_visit_resolved INT     NOT NULL DEFAULT 0,
  avg_star_rating     NUMERIC(3,2), -- Module 19 only
  updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (technician_id, date)
);

CREATE TABLE analytics_business_daily (
  id                UUID      PRIMARY KEY DEFAULT gen_random_uuid(),
  date              DATE      NOT NULL UNIQUE,
  total_jobs        INT       NOT NULL DEFAULT 0,
  installations_completed INT   NOT NULL DEFAULT 0,
  complaints_resolved INT   NOT NULL DEFAULT 0,
  revenue           NUMERIC(14,2) NOT NULL DEFAULT 0,
  pending_payment_total NUMERIC(14,2) NOT NULL DEFAULT 0,
  direct_jobs       INT       NOT NULL DEFAULT 0,
  dealer_jobs       INT       NOT NULL DEFAULT 0,
  repeat_complaint_count INT   NOT NULL DEFAULT 0,
  frequent_customer_count INT   NOT NULL DEFAULT 0,
  avg_resolution_mins NUMERIC(10,2),
  on_time_rate       NUMERIC(5,4),

```

```

    updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE analytics_brand_daily (
    id                  UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    brand_id            UUID          NOT NULL REFERENCES brands(id),
    date                DATE          NOT NULL,
    installations_completed INT        NOT NULL DEFAULT 0,
    complaints_received INT          NOT NULL DEFAULT 0,
    complaints_resolved INT          NOT NULL DEFAULT 0,
    revenue             NUMERIC(14,2) NOT NULL DEFAULT 0,
    revisit_count        INT          NOT NULL DEFAULT 0,
    updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (brand_id, date)
);

CREATE TABLE analytics_dealer_daily (
    id                  UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    dealer_id           UUID          NOT NULL REFERENCES dealers(id),
    date                DATE          NOT NULL,
    jobs_submitted      INT          NOT NULL DEFAULT 0,
    complaints_submitted INT          NOT NULL DEFAULT 0,
    installations_submitted INT        NOT NULL DEFAULT 0,
    resolved_count       INT          NOT NULL DEFAULT 0,
    avg_resolution_mins NUMERIC(10,2),
    pending_count        INT          NOT NULL DEFAULT 0,
    updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (dealer_id, date)
);

CREATE INDEX idx_tech_daily_date ON analytics_technician_daily (date);
CREATE INDEX idx_biz_daily_date ON analytics_business_daily (date);
CREATE INDEX idx_brand_daily_date ON analytics_brand_daily (brand_id, date);
CREATE INDEX idx_dealer_daily_date ON analytics_dealer_daily (dealer_id, date);

```

B.12 — Customer Reviews (Module 19)

```

-- _____
-- CUSTOMER REVIEWS (Optional Module 19)
-- _____

CREATE TABLE customer_reviews (
    id                  UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id              UUID          NOT NULL UNIQUE REFERENCES jobs(id),
    technician_id       UUID          NOT NULL REFERENCES users(id),
    brand_id            UUID          NOT NULL REFERENCES brands(id),
    review_token        UUID          NOT NULL UNIQUE DEFAULT gen_random_uuid(), -- link token
    link_generated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    expires_at          TIMESTAMPTZ NOT NULL,      -- link_generated_at + 48h
    submitted_at        TIMESTAMPTZ,

```

```

    star_rating      SMALLINT CHECK (star_rating BETWEEN 1 AND 5),
    is_lowRated      BOOLEAN GENERATED ALWAYS AS (star_rating <= 2) STORED,
    review_text      TEXT,
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_reviews_technician ON customer_reviews (technician_id);
CREATE INDEX idx_reviews_lowRated ON customer_reviews (is_lowRated) WHERE
is_lowRated = TRUE;

```

C. Relationships

Relationship	Cardinality	Business Logic Mapping
virtual_customers → customer_phones	1 : N	One VCID holds many phone numbers. Lookup by phone resolves to VCID.
virtual_customers → jobs	1 : N	One customer identity can have many jobs over time.
jobs → job_units	1 : N	One job can have multiple AC units at the same address.
jobs → job_assignments	1 : N (1 active)	Assignment history per job. Exactly one row has is_active = TRUE.
jobs → revisits	1 : N	Complaint jobs only. Each Needs Revisit event appends a child revisit record.
revisits → revisit_assignments	1 : N (1 active)	Same pattern as job assignments, scoped to a revisit.
jobs → payments	1 : 1	Exactly one payment record per job. Enforced by UNIQUE on payments.job_id.
jobs → job_timeline	1 : N	Append-only; all events for a job linked here.
jobs → notifications	1 : N	A job can trigger notifications to multiple recipients.
jobs → customer_reviews	1 : 1	Optional Module 19. One review link per completed/resolved job.
users → job_assignments	1 : N	A technician can be assigned to many jobs.
dealers → jobs	1 : N	A dealer can submit many jobs.
dealers → dealer_brands	N : M via dealer_brands	Dealers are associated with one or more brands.
brands → jobs	1 : N	Every job carries a brand. Brand is mandatory.
payment_methods → payments	1 : N	Owner-managed list; referenced by each payment record.

Key design decisions:

- Customer PII (name, address) is embedded in each job rather than in a separate customers table — consistent with the document's "no separate customer table" requirement.

- The VCID (virtual_customers.id) is the sole identity bridge across jobs. It is never surfaced in UI responses.
- Assignment history uses a single table with a partial unique index on is_active = TRUE, enabling O(1) current-technician lookup while preserving full history.
- Revisits are child records of jobs, not independent jobs — preventing lifecycle confusion and enabling the parent job's chronic-complaint flag.

D. Indexing Strategy

Index Target	Type	Justification
customer_phones.phone	B-tree	Phone-number lookup for VCID resolution on every new job entry.
jobs (vcid, created_at DESC)	B-tree composite	All jobs for a customer in chronological order — repeat/frequent detection queries.
jobs (status, type)	B-tree composite	Dashboard job-list filtering by status and type.
jobs (dealer_id, created_at DESC)	B-tree composite	Dealer history view.
jobs (brand_id)	B-tree	Brand-level analytics and filtering.
jobs (scheduled_at)	B-tree	Scheduling conflict window checks.
jobs (is_deleted)	Partial (WHERE is_deleted = FALSE)	Excludes soft-deleted rows from all operational queries.
job_assignments (job_id) WHERE is_active	Partial unique	Enforces single active assignment and supports O(1) current-technician lookup.
job_timeline (job_id, occurred_at DESC)	B-tree composite	Efficient ordered retrieval of all events for a job.
notifications (recipient_user_id, is_read, created_at DESC)	B-tree composite	Notification centre badge count and unread list.
analytics_*_daily (date)	B-tree	Date-range dashboard queries on all analytics tables.
customer_reviews.is_low Rated WHERE TRUE	Partial B-tree	Owner dashboard low-rating flag queries.

Partial Index Note

Partial indexes on is_active = TRUE and is_deleted = FALSE are critical. They keep index sizes small and ensure operational queries never touch soft-deleted or superseded rows.

E. State & Lifecycle Enforcement

Status transitions are enforced in two layers: the NestJS service layer (primary) and a PostgreSQL trigger (safety net). This keeps transition logic in version-controlled code while preventing any direct SQL bypass.

```
-- =====
-- VALID TRANSITION MAP (service layer – not stored in DB)
-- =====

-- Installation transitions (forward only for technicians):
-- pending_schedule → scheduled      (office staff / owner)
-- scheduled        → assigned        (office staff / owner)
-- assigned         → acknowledged    (technician)
-- acknowledged     → in_transit      (technician)
-- in_transit       → in_process      (technician)
-- in_process       → completed        (technician – atomic with payment)
-- * → cancelled      (office staff / owner, pre-completion)

-- Complaint transitions:
-- new              → assigned          (office staff / owner)
-- assigned         → acknowledged      (technician)
-- acknowledged    → in_transit        (technician)
-- in_transit      → in_process        (technician)
-- in_process      → resolved           (technician – atomic with payment)
-- in_process      → needs_revisit      (technician)
-- needs_revisit   → revisit_scheduled (office staff / owner)
-- revisit_scheduled → resolved_on_revisit (technician – atomic with payment)
-- revisit_scheduled → needs_revisit   (technician – loops)
-- * → cancellation_requested (dealer, post-initial state)
-- cancellation_requested → cancelled   (office staff / owner – approve)
-- cancellation_requested → <previous> (office staff / owner – reject)
-- * → cancelled      (office staff / owner direct cancel)

-- Enforcement approach:
-- 1. Store allowed transitions in a constant map in the NestJS service layer.
-- 2. On every status-update request, look up (currentStatus, requestedStatus,
--    actorRole).
--    Reject with HTTP 422 if the transition is not allowed.
-- 3. PostgreSQL CHECK constraint on jobs.status ensures only known status strings
--    are ever stored (typed per job type using a DB function if needed).
-- 4. The DB does NOT store the full transition graph – that belongs in code,
--    where it can be unit-tested, versioned, and reviewed as business logic.

-- Lightweight DB-level guard (catches bugs if service layer is bypassed):
CREATE OR REPLACE FUNCTION validate_job_status()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    valid_installation TEXT[] := ARRAY[
        'pending_schedule', 'scheduled', 'assigned', 'acknowledged',
        'in_transit', 'in_process', 'completed', 'cancelled'
    ];
    valid_complaint TEXT[] := ARRAY[
        'new', 'assigned', 'acknowledged', 'in_transit', 'in_process',
        'resolved', 'needs_revisit', 'revisit_scheduled', 'resolved_on_revisit',
        'cancellation_requested', 'cancelled'
    ];
```

```

];
BEGIN
  IF NEW.type = 'installation' AND NOT (NEW.status = ANY(valid_installation)) THEN
    RAISE EXCEPTION 'Invalid installation status: %', NEW.status;
  END IF;
  IF NEW.type = 'complaint' AND NOT (NEW.status = ANY(valid_complaint)) THEN
    RAISE EXCEPTION 'Invalid complaint status: %', NEW.status;
  END IF;
  RETURN NEW;
END;
$$;

CREATE TRIGGER trg_validate_job_status
  BEFORE INSERT OR UPDATE OF status ON jobs
  FOR EACH ROW EXECUTE FUNCTION validate_job_status();

```

Design Principle

The transition graph lives in the service layer — not in a DB state-machine table. This makes transitions easy to unit-test, diff in PRs, and modify without a migration. The DB trigger is a thin guard that catches programmer errors, not the authoritative logic.

F. Audit & History Design

All significant actions are written to `job_timeline` as immutable append-only rows. The JSONB `previous_value` / `new_value` columns provide a flexible, extensible payload without requiring a column per event type. Rows are never updated or deleted. Soft-deleted jobs remain fully auditable.

```

-- -----
-- EXAMPLE TIMELINE INSERTS
-- -----

-- 1. Status transition
INSERT INTO job_timeline (job_id, event_type, actor_user_id, previous_value,
new_value)
VALUES (
  :job_id, 'status_transition', :user_id,
  '{"status": "in_process"}'::jsonb,
  '{"status": "completed"}'::jsonb
);

-- 2. Payment edit (office staff corrects method)
INSERT INTO job_timeline (job_id, event_type, actor_user_id, previous_value,
new_value)
VALUES (
  :job_id, 'payment_edited', :user_id,
  '{"field": "payment_method", "value": "Cash"}'::jsonb,
  '{"field": "payment_method", "value": "Bank Transfer"}'::jsonb
);

```

```

-- 3. Customer link dismissed
INSERT INTO job_timeline (job_id, event_type, actor_user_id, previous_value,
reason)
VALUES (
:job_id, 'customer_link_dismissed', :user_id,
 '{"matched_customer": "Ahmed Khan"}'::jsonb,
 'Customer link suggestion dismissed – Ahmed Khan – office_staff – 2024-06-
01T09:15:00Z'
);

-- 4. Scheduling conflict acknowledged
INSERT INTO job_timeline
(job_id, event_type, actor_user_id, related_job_id, reason)
VALUES (
:job_id, 'scheduling_conflict_ack', :user_id, :conflicting_job_id,
 'Conflict acknowledged. Assignment proceeded.'
);

-- 5. Owner void on status reversal
INSERT INTO job_timeline (job_id, event_type, actor_user_id, previous_value,
reason)
VALUES (
:job_id, 'payment_void', :owner_id,
 '{"amount": 5000.00, "method": "Cash", "status": "collected"}'::jsonb,
 'Job reopened by owner; payment voided. Reason: incorrect job, relogging
required.'
);

```

Event Type	Captured Fields
status_transition	previous status, new status, actor, timestamp
status_undo / rollback	reverted status, reason, actor, timestamp
assignment / reassignment	previous technician, new technician, actor, timestamp
scheduling_conflict_ack	conflicting job ID, actor, timestamp
payment_recorded / edited	previous value, new value, field, actor, timestamp
payment_void / retained	decision, reason (if void), actor, timestamp
cancellation events	reason, actor / dealer, timestamp
customer_linked / dismissed	matched customer name, actor, timestamp
note_added	note text, actor, timestamp

G. VCID — Virtual Customer Identity Design

The VCID system provides persistent identity linkage across multiple jobs and phone numbers without a traditional customer master record. All PII stays in the jobs table. The VCID is resolved silently in the backend.

```
--
```

```

-- VCID RESOLUTION (service layer pseudocode expressed as SQL steps)
-- _____

-- Step 1: Check incoming phone number against customer_phones.
SELECT cp.vcid, vc.id, j.customer_name, j.address
FROM   customer_phones cp
JOIN   virtual_customers vc ON vc.id = cp.vcid
JOIN   jobs j ON j.vcid = vc.id
WHERE  cp.phone = :incoming_phone
ORDER  BY j.created_at DESC
LIMIT  1;

-- If match found → prompt office staff "Link to existing customer: [name,
address]?"

-- Step 2a: Staff confirms link → job is created with the existing vcid.
--   If the incoming phone is new, add it to customer_phones:
INSERT INTO customer_phones (vcid, phone, linked_by)
VALUES (:existing_vcid, :incoming_phone, :user_id)
ON CONFLICT (vcid, phone) DO NOTHING;

-- Step 2b: Staff dismisses → new VCID created, dismissed event logged in
job_timeline.
INSERT INTO virtual_customers DEFAULT VALUES RETURNING id AS new_vcid;
-- job.vcid = new_vcid

-- Step 3 (name + address partial match – secondary signal):
SELECT DISTINCT cp.vcid, j.customer_name, j.address
FROM   jobs j
JOIN   customer_phones cp ON cp.vcid = j.vcid
WHERE  j.customer_name ILIKE '%' || :name_fragment || '%'
AND    j.address ILIKE '%' || :address_fragment || '%'
LIMIT  5;
-- Same link / dismiss flow applies.

-- _____
-- REPEAT AND FREQUENT COMPLAINT DETECTION
-- _____

-- Called when a new complaint job is saved.
SELECT COUNT(*) AS cnt
FROM   jobs
WHERE  vcid = :vcid
AND    type = 'complaint'
AND    created_at >= NOW() - (:window_days || ' days')::INTERVAL
AND    is_deleted = FALSE;
-- If cnt >= 1 within repeat_complaint_window_days → flag job.is_repeat = TRUE

SELECT COUNT(*) AS cnt
FROM   jobs
WHERE  vcid = :vcid

```

```

AND type = 'complaint'
AND created_at >= NOW() - '90 days'::INTERVAL
AND is_deleted = FALSE;
-- If cnt >= :frequent_threshold → flag job.is_frequent = TRUE,
-- set virtual_customers.is_frequent = TRUE, fire notification.

```

Scenario	System Behaviour
Returning customer, same phone	Phone matches customer_phones → VCID resolved automatically → link prompt shown.
Returning customer, new phone	No phone match; name+address partial match → link prompt shown → staff confirms → new phone added to customer_phones under existing VCID.
Prompt dismissed	New VCID generated. Dismissal logged in job_timeline. Staff can manually re-link later.
Dealer-submitted job	Same resolution flow. Phone checked server-side on submission; VCID linked or created.
Repeat complaint check	Query jobs by VCID within configurable window. Operates on VCID — not raw phone — so accuracy holds across multiple contact numbers.

H. Concurrency & Data Integrity

```

-- -----
-- OPTIMISTIC LOCKING PATTERN
-- -----
-- jobs.version and payments.version increment on every UPDATE.
-- The service layer reads version before updating and passes it back.
-- If version has changed by another actor, the UPDATE returns 0 rows
-- and the service raises a 409 Conflict.

UPDATE jobs
SET
  status      = :new_status,
  updated_at = NOW(),
  version     = version + 1
WHERE id      = :job_id
  AND version = :expected_version -- must match what was read
  AND is_deleted = FALSE;

-- If rowcount = 0 → raise OptimisticLockException → return HTTP 409

-- -----
-- PAYMENT + STATUS ATOMIC PAIR (sync queue pattern)
-- -----
-- Both submitted as one database transaction from the sync worker:

BEGIN;

```

```

UPDATE jobs
SET status = 'completed', version = version + 1, updated_at = NOW()
WHERE id = :job_id AND version = :v;

INSERT INTO payments (job_id, amount, payment_method_id, status, recorded_by)
VALUES (:job_id, :amount, :method_id, 'collected', :technician_id);

INSERT INTO job_timeline (job_id, event_type, actor_user_id, new_value)
VALUES (:job_id, 'status_transition', :technician_id,
'{"status":"completed"}'::jsonb);

INSERT INTO job_timeline (job_id, event_type, actor_user_id, new_value)
VALUES (:job_id, 'payment_recorded', :technician_id,
        json_build_object('amount', :amount, 'method_id', :method_id)::jsonb);
COMMIT;

```

Why Optimistic, Not Pessimistic Locking?

Pessimistic locking (SELECT FOR UPDATE) risks deadlocks and degraded throughput under concurrent field-sync events. Optimistic locking using a version counter is sufficient for this workload: conflicts are rare, and the 60-second undo window makes re-submission by the technician straightforward.

I. Analytics Support

Analytics are precomputed into four daily rollup tables (analytics_technician_daily, analytics_business_daily, analytics_brand_daily, analytics_dealer_daily). Each job mutation triggers targeted UPSERT operations on the relevant rows. Dashboard queries never touch the jobs table directly.

```

-- =====
-- ANALYTICS UPDATE TRIGGER (fires after each job event)
-- =====
-- A lightweight trigger function called by NestJS after any job mutation.
-- Performs UPSERT on the relevant daily rollup row(s).

-- Example: update technician daily row when a complaint is resolved
INSERT INTO analytics_technician_daily
(technician_id, date, complaints_resolved, total_payment_collected)
VALUES
(:technician_id, CURRENT_DATE, 1, :amount)
ON CONFLICT (technician_id, date) DO UPDATE
SET complaints_resolved = analytics_technician_daily.complaints_resolved + 1,
    total_payment_collected = analytics_technician_daily.total_payment_collected
+ :amount,
    updated_at = NOW();

-- Business-wide rollup for revenue charts
INSERT INTO analytics_business_daily (date, revenue, complaints_resolved)
VALUES (CURRENT_DATE, :amount, 1)
ON CONFLICT (date) DO UPDATE

```

```

SET revenue          = analytics_business_daily.revenue + :amount,
  complaints_resolved = analytics_business_daily.complaints_resolved + 1,
  updated_at          = NOW();

-- =====
-- DASHBOARD QUERY EXAMPLES (hit precomputed tables – sub-millisecond)
-- =====

-- Monthly revenue (last 12 months)
SELECT DATE_TRUNC('month', date) AS month,
       SUM(revenue) AS total_revenue
FROM   analytics_business_daily
WHERE  date >= NOW() - INTERVAL '12 months'
GROUP BY 1
ORDER BY 1;

-- Technician leaderboard (current month)
SELECT u.full_name,
       SUM(d.complaints_resolved + d.installations_completed) AS total_jobs,
       SUM(d.total_payment_collected) AS revenue,
       AVG(d.first_visit_resolved::FLOAT /
           NULLIF(d.complaints_resolved,0)) AS first_visit_rate
FROM   analytics_technician_daily d
JOIN   users u ON u.id = d.technician_id
WHERE  d.date >= DATE_TRUNC('month', NOW())
GROUP BY u.id, u.full_name
ORDER BY revenue DESC;

```

Analytics Feature	Source Table	Notes
Revenue by month	analytics_business_daily	SUM(revenue) grouped by month.
Technician leaderboard	analytics_technician_daily	Ranked by jobs, revenue, first-visit rate.
Per-brand metrics	analytics_brand_daily	Installs, complaints, revenue, revisit rate.
Dealer leaderboard	analytics_dealer_daily	Volume and resolution rate per dealer.
Repeat / frequent customers	jobs + virtual_customers	Queried on VCID; flags cached on vc.is_frequent.
Busiest days / hours	analytics_business_daily + jobs	Hour-of-day breakdown from jobs.created_at.
First-visit resolution rate	analytics_technician_daily	first_visit_resolved / complaints_resolved.
On-time arrival rate	analytics_business_daily / tech	on_time_count / (on_time + late).
CSV export	All analytics tables	Role-scoped SELECT query; streamed to CSV by service layer.